

Cours 1 ECABD MAABOUT

Associations

- Un hypermarché peut savoir quels sont les produits qui ont été achetés simultanément lors d'un passage en caisse
- Quels sont les produits fréquemment achetés ensemble -> **Association entre produits**
- Si du **Lait** et du **Beurre** sont associés alors pas besoin de mettre une promotion sur ces 2 produits en même temps.

Classifications

- Une banque dispose de l'historique des remboursements des crédits de ses clients
 - Une « Analyse » de cet historique conjointement avec le profil des clients permet d'identifier les **critères** qui font qu'un client soit considéré (**classé**) comme **bon** ou **mauvais**
 - Un nouveau client qui demande un crédit, on peut « **prédire** » sa classe et donc décider de lui attribuer le crédit ou pas.
- ➔ On veut trouver un modèle ou manière de prendre une décision en fonction de l'historique d'une telle personne ou action

Clustering (Regroupement)

- Grâce aux cartes de fidélité, un hypermarché peut construire des **groupes** de clients qui ont des **comportements similaires** relativement à leurs achats.
- On peut donc faire des **campagnes de publicité ciblées** en proposant des parties du catalogue général ne contenant que les produits qui intéressent chacun des groupes.
- Deux clients appartiennent au même groupe si jamais ils font le même type d'achat.

Règles d'Association

Motifs de la forme : Corps -> Tête

Ex : achète(x, « cacahuètes ») -> achète(x, « bières »)

Etant donnés :

- (1) Une base de transactions
- (2) Chaque transaction est décrite par un identifiant et une liste d'items

Objectif : Trouver les règles qui expriment une corrélation entre la présence d'un item avec la présence d'un ensemble d'items

→ **Ex :** 98% des personnes qui achètent des cacahuètes achètent de la bière

Mesure : Support et Confiance

Pour juger de l'intérêt d'une règle d'**Association**, on doit mesurer le support et la confiance.

On a un taux de support minimal à fixer, qui fait en sorte que l'association soit intéressante à considérer

- **Support de cette règle :** $X \& Y \Rightarrow Z$ (Il faut que sur les tickets, il y'ait X, Y et Z)
→ Pour que ce support soit intéressant, il faut qu'il soit supérieur au support minimal fixé
→ **Remarque :** $X \& Y \Rightarrow Z$ aura la même règle que $X \& Z \Rightarrow Y$ (!\ pas pareil pour la **confiance**)
- **Confiance de cette règle :** Probabilité conditionnelle qu'une transaction qui contient **X&Y** contiennent **aussi Z** -> **Confiance=Support(X,Y,Z)/Support(X,Y)**

ID Transaction	Items
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Soit support minimum 50%, et
confiance minimum 50%,
 $A \Rightarrow C$ (50%, 66.6%)
 $C \Rightarrow A$ (50%, 100%)

Pour qu'une règle soit intéressante, il faut que cette règle soit présent dans au moins 50% des transactions

Exemple d'application :

- **Support(A->C)**
 - Fréquence des transactions qui contiennent A et C
 - 2/4 -> 50%
- **Confiance(A->C)**
 - Fréquence des transactions contenant A qui contiennent également C
 - 2/3 -> 66%
- **Support(C->A)**
 - Fréquence des transactions qui contiennent A et C
 - 2/4 -> 50%
- **Confiance(C->A)**
 - Fréquence des transactions contenant C qui contiennent également A
 - 2/2 -> 100%

→ Pour détecter ces règles automatiquement, il va falloir établir des algorithmes d'extraction efficaces

Extraction des transactions : Première Approche

Objectif : Extraction des ensembles fréquents

```
D : base de transactions (ensemble de transactions)
I : ensemble de tous les items (|I| = n)

Fréquent ← ∅ // ensemble des itemsets fréquents trouvés

Pour chaque J ⊆ I : // pour chaque sous-ensemble J de l'ensemble I (tel que J est un sous-ensemble de produits -> EX : J={Beurre},{Pain,Beurre},...)
    Count(J) ← 0
    Pour chaque transaction t ∈ D : //On parcourt toutes les transactions dans notre base
        Si J ⊆ t.items : // si tous les items de J sont présents dans la transaction t
            Count(J) ← Count(J) + 1
    Fin du pour //Une fois qu'on a exploré toutes les transactions, on vérifie si notre règle est "fréquente"
    Si Count(J) ≥ min_support : // min_support = seuil (absolu ou relatif) fixé
        ajouter J à Fréquent //On ajoute notre sous-ensemble dans la liste Fréquent
Fin Pour
```

Pour voir l'algo en plus grand, cf **Algo1_Association.txt**

Extraction des transactions : Première Approche

Objectif : Extraction des règles

```
D : base de transactions
I : ensemble de tous les items (|I| = n)

Algorithme 2 : Extraction des règles d'association
-----
Règles ← ∅ // ensemble vide de règles valides

// Étape : parcourir les itemsets fréquents trouvés avec l'algorithme 1
Pour chaque J ∈ Fréquent : // J est un itemset fréquent

    // Étape : générer toutes les règles possibles à partir de J -> Partitionner J = Séparer les items de J en plusieurs sous ensembles
    -> Pour chaque façon de couper J en deux parties (A et B), on fabrique une règle A ⇒ B.
    Pour chaque partition de J en (A, B) telle que J = A ∪ B, A ∩ B = ∅ :
        règle r : A ⇒ B

        // Calcul de la confiance de la règle
        confiance(r) = support(J) / support(A)

        // Vérification du seuil
        Si confiance(r) ≥ min_confiance :
            ajouter r à Règles

Fin
/*-----*/

Exemple avec J={Pain,Lait,Beurre}

Voici toutes les partitions possibles :

Un élément seul à droite :

A = {Pain, Lait}, B = {Beurre} → règle : {Pain, Lait} ⇒ {Beurre}
A = {Pain, Beurre}, B = {Lait} → règle : {Pain, Beurre} ⇒ {Lait}
A = {Lait, Beurre}, B = {Pain} → règle : {Lait, Beurre} ⇒ {Pain}

Deux éléments à droite (équivalent à un seul à gauche) :

A = {Pain}, B = {Lait, Beurre} → règle : {Pain} ⇒ {Lait, Beurre}
A = {Lait}, B = {Pain, Beurre} → règle : {Lait} ⇒ {Pain, Beurre}
A = {Beurre}, B = {Pain, Lait} → règle : {Beurre} ⇒ {Pain, Lait}
```

Complexité globale de ces deux algos : **2ⁿ parcours de D** (tel que n = nombre d'items de I)

Extraction des associations : A priori

Principe : Si un ensemble est **non-fréquent**, alors **tous** ses **sur-ensembles** ne sont pas fréquents.

→ **Exemple** : Si $\{A\}$ n'est pas fréquent, alors $\{AB\}$ ne peut pas l'être -> Donc pas la peine de les tester

ID Transaction	Items
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Soit support minimum 50%, et
confiance minimum 50%,
 $A \Rightarrow C$ (50%, 66.6%)
 $C \Rightarrow A$ (50%, 100%)

Ici E n'est pas fréquent (support < 50%) donc $\{EA\}$ est forcément pas fréquent

- Si $\{AB\}$ est fréquent, alors $\{A\}$ et $\{B\}$ sont fréquents

Objectif :

1. Ensemble d'items de cardinalité 1 -> On compte la fréquence de chaque item seul
→ Si un item n'est pas fréquent, pas besoin de le considérer par la suite
2. Ensemble d'items de cardinalité 2 -> On compte les fréquents de taille 1 pour compter ceux de taille 2
3. Ensemble d'items de cardinalité 3 -> etc...

```
Algorithme A Priori
Entrées :
  D = base de transactions
  min_support = seuil minimal de support

Sortie :
  L = ensemble de tous les itemsets fréquents

Début
  # Étape 1 : trouver les 1-itemsets fréquents
  L1 = { tous les items uniques dont le support >= min_support }

  k = 1
  Tant que Lk n'est pas vide faire : #Lk est la liste des ensembles d'items de taille k

    # Étape 2 : génération des candidats
    # On joint les itemsets fréquents de taille k pour générer
    # des candidats de taille k+1
    Ck+1 = GénérerCandidats(Lk)

    # Étape 3 : comptage des supports
    Pour chaque transaction t dans D faire :
      Pour chaque candidat c dans Ck+1 faire : #Un candidat est un ensemble d'items que l'on veut examiner
        Si c ⊆ t alors #Si notre sous ensemble est présent dans la transaction alors on incrémente notre compte
          incrémenter count(c) # c apparaît dans t
        FinSi
      FinPour
    FinPour

    # Étape 4 : filtrage
    Lk+1 = { c ∈ Ck+1 | support(c) >= min_support }

    # Passage à la taille suivante
    k = k + 1

  FinTantQue

  # Étape 5 : retourner tous les itemsets fréquents
  Retourner ⋃ Lk
Fin
```

```

fonction genererCandidats(Lk-1):           # Déclare une fonction pour générer les candidats de taille k à partir de Lk-1
    Ck = []                               # Initialise la liste vide qui contiendra les candidats de taille k
#Le self-join consiste à combiner des itemsets de taille k-1 entre eux pour créer des candidats de taille k.
    pour chaque itemset p dans Lk-1:      # Parcourt chaque itemset fréquent p de taille k-1
        pour chaque itemset q dans Lk-1:  # Pour chaque p, parcourt chaque itemset q de Lk-1 pour combiner p et q
            # Vérifie que les k-2 premiers items sont identiques et que le dernier item de p < dernier item de q
            si p[1] == q[1] et p[2] == q[2] ... et p[k-2] == q[k-2] et p[k-1] < q[k-1]:
                c = concat(p[1..k-1], q[k-1]) # Crée un candidat c en combinant p et le dernier item de q
                ajouter c à Ck                # Ajoute le candidat c à la liste des candidats Ck
#Le pruning consiste à supprimer les candidats qui ne peuvent pas être fréquents, en utilisant la propriété fondamentale de l'Apriori :
#-> "Si un itemset est fréquent, alors tous ses sous-ensembles sont aussi fréquents."
    pour chaque candidat c dans Ck:        # Parcourt chaque candidat généré pour effectuer le pruning
        pour chaque (k-1)-sous-ensemble s de c: # Génère tous les sous-ensembles de taille k-1 du candidat c
            si s n'est pas dans Lk-1:         # Si un sous-ensemble n'est pas fréquent, alors c ne peut pas être fréquent
                supprimer c de Ck            # Supprime ce candidat de Ck car il ne peut pas être fréquent
                sortir de la boucle sur les sous-ensembles # Pas besoin de tester les autres sous-ensembles

    retourner Ck                           # Renvoie la liste finale des candidats fréquents de taille k

```

La génération de candidats risque d'être extrêmement coûteux et longs, surtout que l'algo part du principe qu'un client peut prendre une infinité de produits...

- Beaucoup:
 - 10^4 1-itemsets fréquents générant 10^7 2-itemsets candidats
 - Pour trouver les 100-itemsets on doit générer $2^{100} \approx 10^{30}$ candidats.
- Plusieurs scans de la base:
 - On doit faire $(n + 1)$ scans, pour trouver les n -itemsets fréquents

Possibilité d'explorer sans la génération de candidats : Fp-tree

Nous allons **compresser** notre bdd afin d'économiser de la mémoire et de coûts. On va utiliser pour cela une **structure arborescente** : le **FP-Tree**.

Chaque chemin de notre arbre correspond à une transaction, mais les branches permettent de partager les **items communs**.

Exemple : (Voir FDR)

Transactions :

1. {A, B, C}
2. {A, B, D}
3. {A, C}

FP-tree :

mathematica Copier le code

```

graph TD
    A[A] --- B1[ ]
    B1 --- B[B]
    B1 --- C1[C]
    B1 --- D[D]
    style B1 width:0px,height:0px
    style B fill:none,stroke:none
    style C1 fill:none,stroke:none
    style D fill:none,stroke:none

```

- Les items communs (comme A) sont **fusionnés**, ce qui économise beaucoup de mémoire et évite de scanner toute la base plusieurs fois.
- 💡 Objectif : **représentation condensée** qui réduit le coût des scans.

Prise de décision : Important de prendre en compte la corrélation afin de ne pas prendre de décisions hâtives. Objectif -> Est-ce que notre règle est intéressante ?

- Intérêt (corrélation) $\frac{P(A \wedge B)}{P(A)P(B)}$
 - Prendre en compte $P(A)$ et $P(B)$
 - $P(A \& B) = P(B) * P(A)$, si A et B sont des événements indépendants
 - A et B négativement corrélés, si $\text{corr}(A, B) < 1$.

X	1	1	1	1	0	0	0	0
Y	1	1	0	0	0	0	0	0
Z	0	1	1	1	1	1	1	1

Itemset	Support	Intérêt
X,Y	25%	2
X,Z	37,50%	0,9
Y,Z	12,50%	0,57